

LAPORAN PRATIKUM

MODULE 3

Dosen pengampu : Dr.Ir Septafiansyah Dwi
Putra,S.T.,MT

Agus ambarwari,S.Pd.,M.Kom



Disusun oleh:

Nama : Mutia Qauna Hidayah

Npm : 25782048

Kelas : TRI 3B

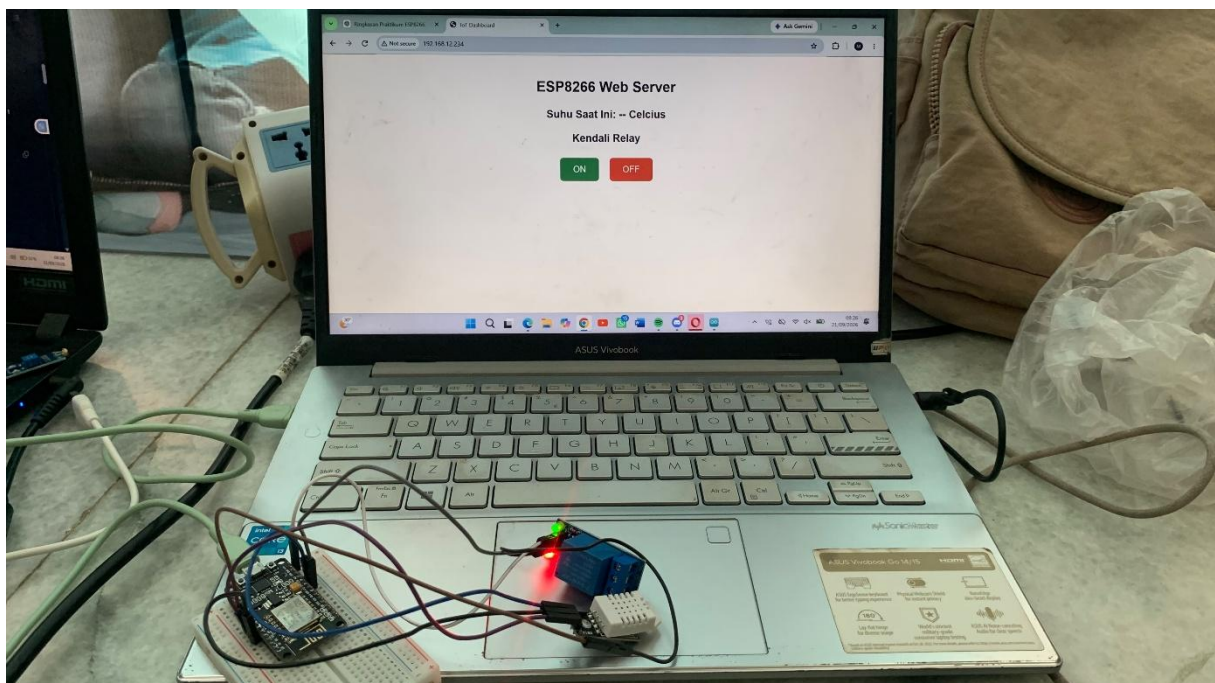
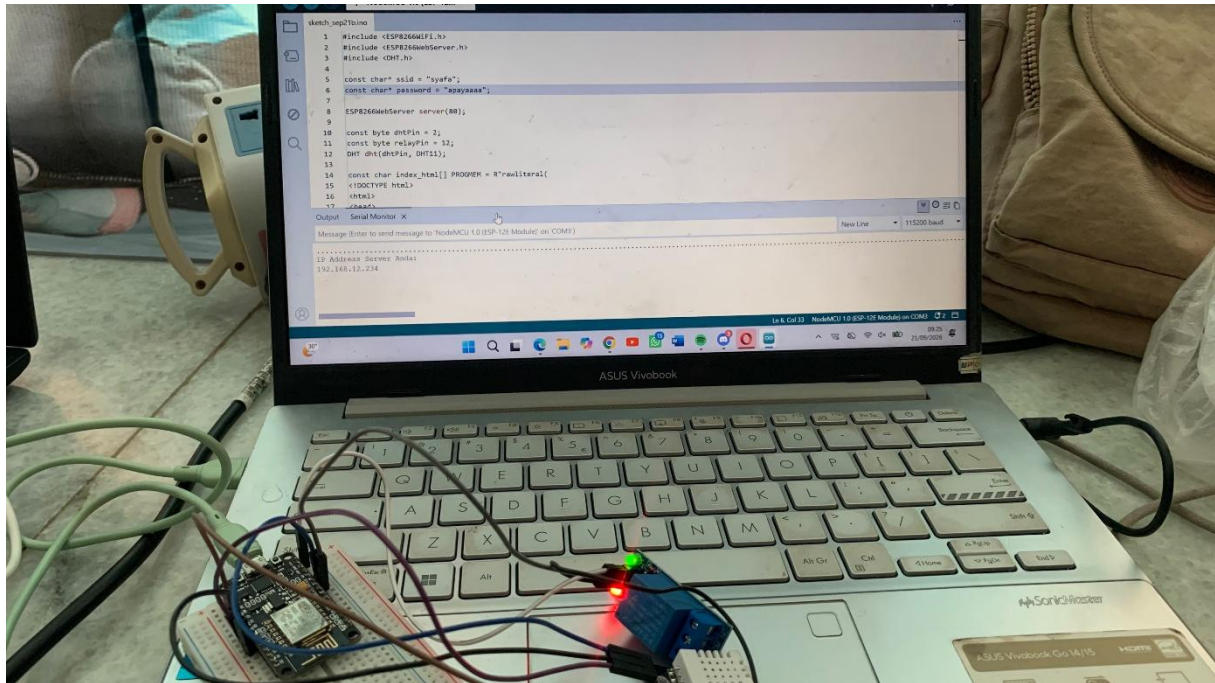
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG TAHUN 2026

Praktikum 1

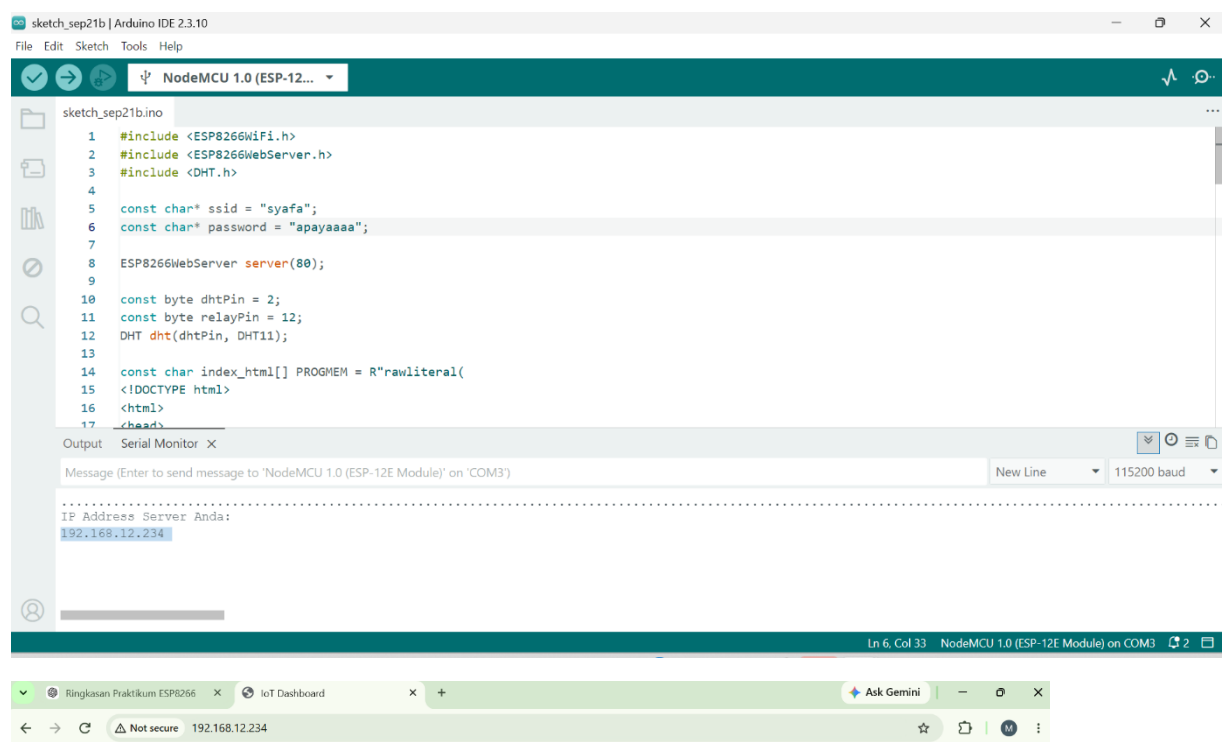
3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)



Pada praktikum ini, NodeMCU ESP8266 berfungsi sebagai Web Server yang terhubung ke jaringan Wi-Fi. Sensor DHT11 digunakan untuk membaca suhu ruangan, kemudian hasilnya ditampilkan pada halaman web. Web Server juga menyediakan tombol ON dan OFF untuk mengendalikan LED atau

Relay melalui browser. Praktikum ini menunjukkan bahwa perangkat IoT dapat dikendalikan melalui jaringan Wi-Fi menggunakan HTTP.

OUTPUT KODE NYA



The screenshot displays the Arduino IDE environment with the sketch 'sketch_sep21b.ino' open. The code is for an ESP8266 Web Server that includes libraries for WiFi, WebServer, and DHT. It sets up a server on port 80 and a DHT sensor. The web page content is defined in the 'index_html' array. Below the code editor, the Serial Monitor shows the output: 'IP Address Server Anda: 192.168.12.234'. Below the IDE, a web browser window is open at the IP address 192.168.12.234, displaying the web server's interface.

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "syafa";
6 const char* password = "apayaaaa";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
```

Output: IP Address Server Anda: 192.168.12.234

Web Browser Output:

ESP8266 Web Server

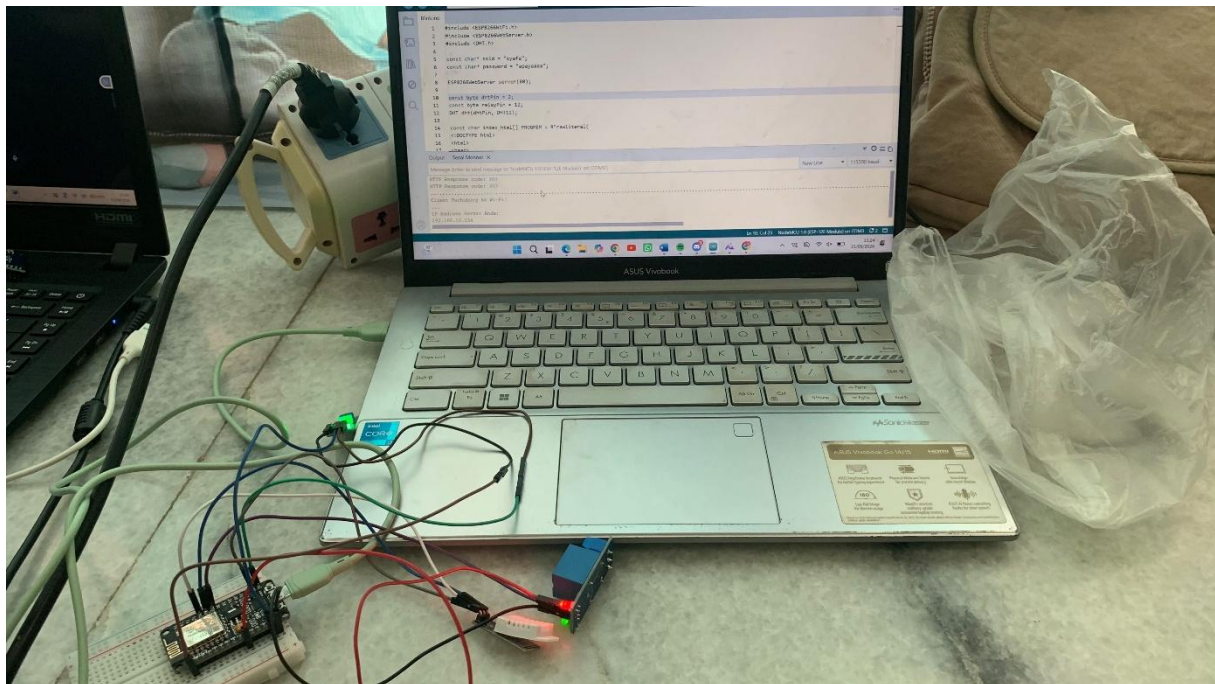
Suhu Saat Ini: **1.80** Celcius

Kendali Relay

ON OFF

Hasil yang diharapkan: Dashboard dapat dibuka melalui IP Address NodeMCU, suhu ditampilkan pada browser, dan tombol ON/OFF dapat mengendalikan output.

3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)



Pada praktikum ini, NodeMCU ESP8266 berfungsi sebagai HTTP Client yang membaca intensitas cahaya menggunakan sensor LDR. Ketika kondisi gelap terdeteksi (nilai ADC di bawah 300), Client mengirim HTTP GET ke alamat Web Server melalui jaringan Wi-Fi. Server kemudian menjalankan perintah untuk menyalakan LED atau Relay. Praktikum ini menunjukkan komunikasi antarperangkat IoT secara nirkabel.

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "syafa";
6 const char* password = "apayaaaa";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

HTTP Response code: 303
HTTP Response code: 303
.....
Client Terhubung ke Wi-Fi!
...
IP Address Server Anda:
192.168.12.234

Hasil yang diharapkan: Ketika LDR ditutup hingga gelap, Client mengirim permintaan HTTP dan Server menerima perintah untuk menyalakan output.

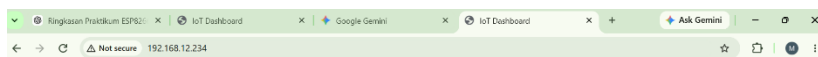
3.6 Latihan

- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Mode Station (STA) digunakan karena NodeMCU pada kedua praktikum berperan sebagai perangkat yang terhubung ke jaringan Wi-Fi yang sama. Dengan mode STA, NodeMCU dapat berkomunikasi dengan perangkat lain, seperti Client dan Server, melalui jaringan lokal. Sedangkan mode Access Point (AP) membuat NodeMCU menjadi pemancar Wi-Fi sendiri, sehingga tidak sesuai dengan skenario praktikum yang membutuhkan kedua NodeMCU terhubung ke satu jaringan bersama.

Latihan 2 (Modifikasi Antarmuka & Telemetri): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

- i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.
- ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi **LED** saat ini:
 - o Jika LED sedang **OFF**, tombol menampilkan tulisan **"NYALAKAN (ON)"** (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`.
 - o Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan **"MATIKAN (OFF)"** (warna merah) dan mengarahkan aksi ke `/relay/off`.
(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti `%RELAY_BUTTON%`, lalu di fungsi `handleRoot()`, periksa status pin relay/LED menggunakan `digitalRead(relayPin)` sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang Anda ubah



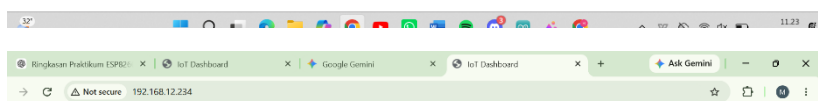
ESP8266 Web Server

Suhu Saat Ini: **1.70** Celcius

Kelembapan: **18.70** %

Kendali Relay

MATIKAN (OFF)



ESP8266 Web Server

Suhu Saat Ini: **1.60** Celcius

Kelembapan: **18.20** %

Kendali Relay

NYALAKAN (ON)



The screenshot shows the Arduino IDE interface with a sketch named 'Blink.ino' for a NodeMCU 1.0 (ESP-12E) module. The code includes libraries for ESP8266 WiFi and HTTP client. It sets up a WiFi connection with SSID 'syafa' and password 'apayaaaaa'. The server URL is 'http://192.168.12.226/relay/on'. In the setup function, the serial port is initialized at 115200 baud, and the WiFi mode is set to STA. The main loop checks if the WiFi is connected; if not, it delays for 500ms and prints a dot. Once connected, it prints 'Client Terhubung ke Wi-Fi!'. The output window shows the serial monitor with the following messages: 'HTTP Response code: 303', 'HTTP Response code: 303', 'Client Terhubung ke Wi-Fi!', and 'IP Address Server Anda: 192.168.12.234'.

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "syafa";
6 const char* password = "apayaaaaa";
7
8 const char* serverName = "http://192.168.12.226/relay/on";
9 const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

HTTP Response code: 303
HTTP Response code: 303
.....
Client Terhubung ke Wi-Fi!
...
IP Address Server Anda:
192.168.12.234

Fungsi `dht.readHumidity()` digunakan untuk membaca kelembapan, sedangkan `html.replace()` mengganti placeholder dengan nilai sensor. Tombol dinamis menyesuaikan kondisi Relay, yaitu menampilkan NYALAKAN (ON) ketika Relay OFF dan MATIKAN (OFF) ketika Relay ON.

Catatan: Logika tombol di atas mengikuti program Praktikum 1, yaitu Relay HIGH = ON dan LOW = OFF

- **Latihan 3 (Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

The screenshot shows the same Arduino IDE interface as the first image, but the output window now displays multiple 'HTTP Response code: -1' messages. This indicates that the client is unable to connect to the server, likely because the server is offline.

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "syafa";
6 const char* password = "apayaaaaa";
7
8 const char* serverName = "http://192.168.12.234/relay/on";
9 const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
```

Output Serial Monitor X

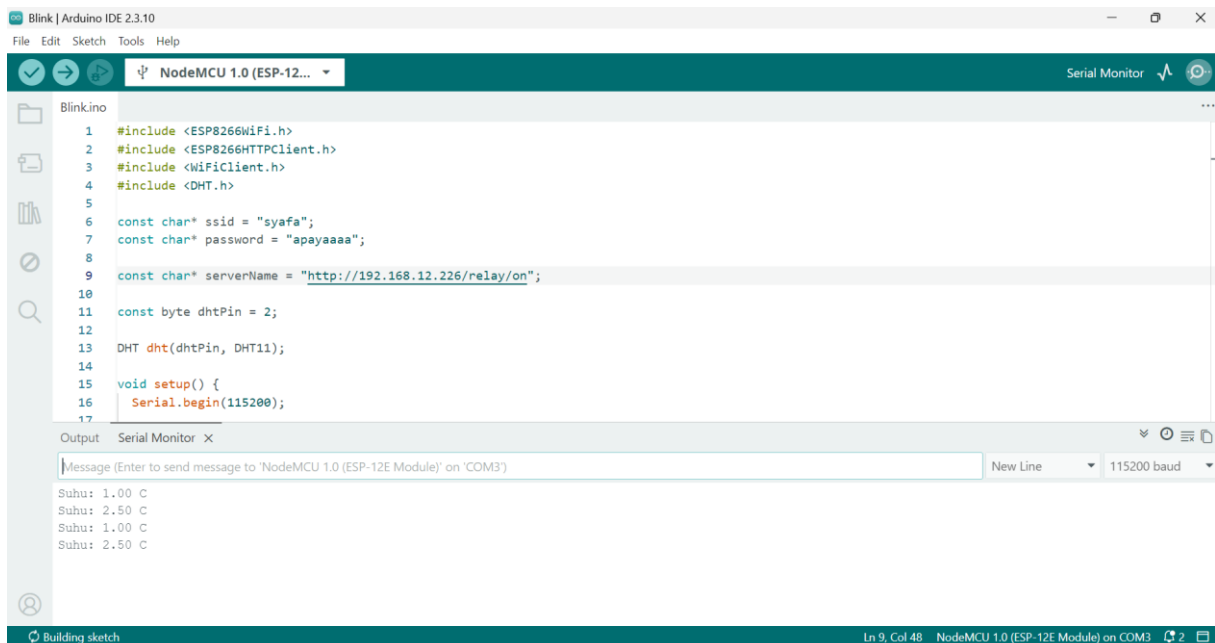
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1

Jika NodeMCU Server mendadak kehilangan daya (*offline*), Client tidak akan mendapatkan respons dari Server. Serial Monitor akan menampilkan HTTP Response code: -1 karena permintaan HTTP

gagal akibat Server tidak dapat dijangkau. Hal ini menunjukkan bahwa komunikasi antara Client dan Server terputus.

- **Latihan 4 (Pengembangan Algoritma):** Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* `if` di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.



```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4 #include <DHT.h>
5
6 const char* ssid = "syafa";
7 const char* password = "apayaaaa";
8
9 const char* serverName = "http://192.168.12.226/relay/on";
10
11 const byte dhtPin = 2;
12
13 DHT dht(dhtPin, DHT11);
14
15 void setup() {
16   Serial.begin(115200);
17 }
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

Suhu: 1.00 C
Suhu: 2.50 C
Suhu: 1.00 C
Suhu: 2.50 C

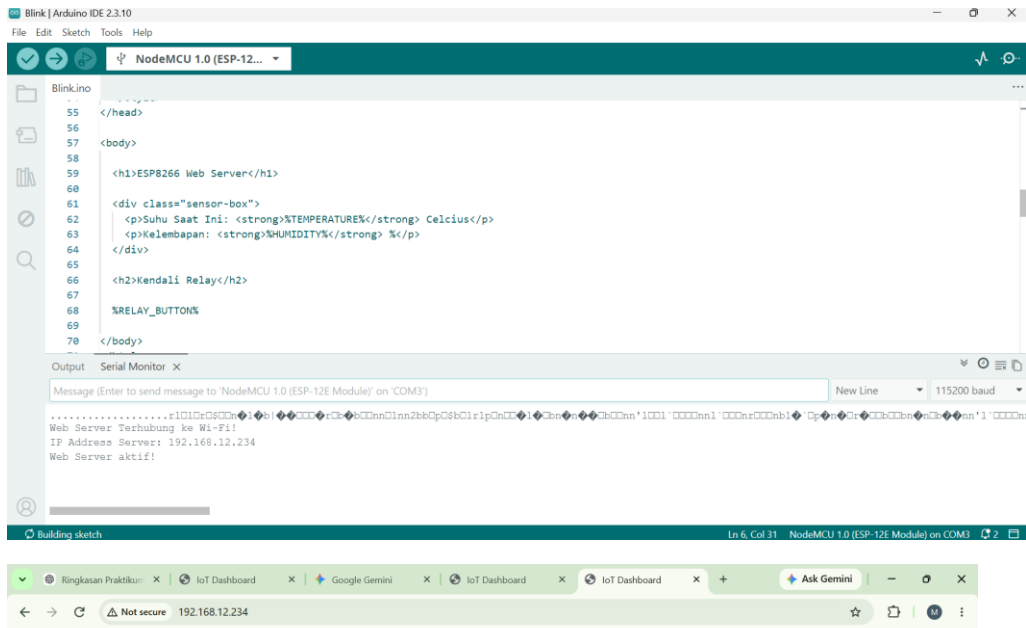
Client hanya mengirim HTTP GET ketika Wi-Fi terhubung, pembacaan suhu berhasil, dan suhu melebihi 35°C. Jika suhu 35°C atau kurang, Client tidak mengirim permintaan ke Server

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Antarmuka (*User Interface*) IoT tidak boleh statis. Saat ini, *Web Server* kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (Web Dashboard): Anda akan bertindak sebagai *Full-Stack IoT Developer* untuk memutakhirkan kapabilitas *Web Server* pada **Latihan 2**, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

- **Tujuan:** Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*).



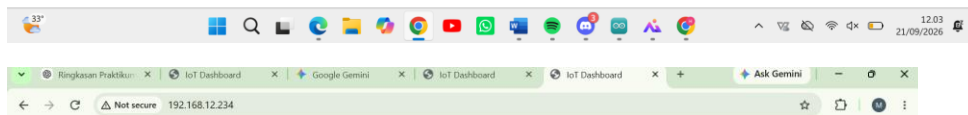
ESP8266 Web Server

Suhu Saat Ini: **2.40** Celcius

Kelembapan: **21.10** %

Kendali Relay

NYALAKAN (ON)



ESP8266 Web Server

Suhu Saat Ini: **2.30** Celcius

Kelembapan: **19.30** %

Kendali Relay

MATIKAN (OFF)

Meta Refresh merupakan cara sederhana untuk membuat Web Dashboard IoT memperbarui data secara otomatis. Namun, metode ini memiliki kelemahan berupa layar berkedip, penggunaan bandwidth yang lebih besar, dan interaksi pengguna yang kurang nyaman. Oleh karena itu, Meta Refresh belum ideal untuk sistem Real-Time Dashboard. Metode seperti AJAX atau WebSocket dapat

menjadi alternatif karena memungkinkan pembaruan data tanpa harus memuat ulang seluruh halaman.